



Feedback SDK

Lightweight in-app user feedback SDK

A lightweight solution for Android developers to collect structured user feedback, ratings, suggestions, and optional screenshots with minimal integration effort. The SDK adds useful app and device context so product teams can better understand user opinions, suggestions, and experience inside the app.

 Ido Ben Bassat

Main Features and Capabilities



In-App Feedback Interface

Native, non-intrusive dialog for collecting user feedback directly within the host application.



Contextual Screenshots

User-approved visual context that helps the product team understand what the user was seeing when submitting feedback.



Structured Feedback Collection

Allows users to submit comments, suggestions, complaints, praise, and usability feedback in a consistent format.



Automatic Context Collection

Collects useful device and app context, such as device model, Android version, app version, and screen metadata.



User Experience Ratings

Quantitative 1–5 star assessment to accurately track and measure user sentiment.



Firebase Integration

Seamlessly utilizes Firestore for structured data and Storage for handling visual assets.

Public SDK API

init

Initializes the SDK with the application context and API key. This is called once in the Application class to set up SDK configuration and validate the API key.

showFeedbackDialog

Opens the built-in feedback dialog UI where the user can submit a rating, suggestion, complaint, praise, or general app feedback.

setUser

Allows the host app to attach optional user identity information, such as user ID or email, to feedback items for follow-up communication.

setMetadata

Allows the developer to attach custom key-value metadata, such as current screen, subscription type, environment, or feature flag status.

Internal SDK Logic

collectDeviceInfo

Collects manufacturer, model, Android version, locale, and screen dimensions.

collectAppInfo

Extracts package name, app version name, version code, and build type.

captureScreenshot

Captures the current Activity view only after the user confirms screenshot attachment.

compressScreenshot

Compresses the screenshot image file locally to reduce upload size and time.

validateFeedbackInput

Validates the feedback message, rating, optional email, and required metadata.

buildFeedbackPayload

Builds the structured feedback object before upload.

sendFeedbackToServer

Sends the structured feedback payload to the backend.

uploadScreenshot

Uploads the compressed screenshot to Firebase Storage when the user chooses to attach one.

clearTemporaryFiles

Deletes local cached screenshots post-upload to free up user device storage.

Backend & Server Logic

validateApiKey

Checks that the request API key belongs to a registered and active app.

validateFeedbackPayload

Ensures incoming payload matches schema requirements and sanitizes inputs.

createFeedbackItem

Validates the payload, creates the main feedback item object, and prepares for storage.

uploadScreenshotToStorage

Uploads the compressed screenshot to Firebase Storage and returns a storage path or authorized download URL.

saveFeedbackToFirestore

Commits the validated feedback document into the specific app collection.

getFeedbackByAppId

Fetches all user feedback items associated with a specific tenant application.

getFeedbackById

Retrieves the full detailed document for a single specific feedback item.

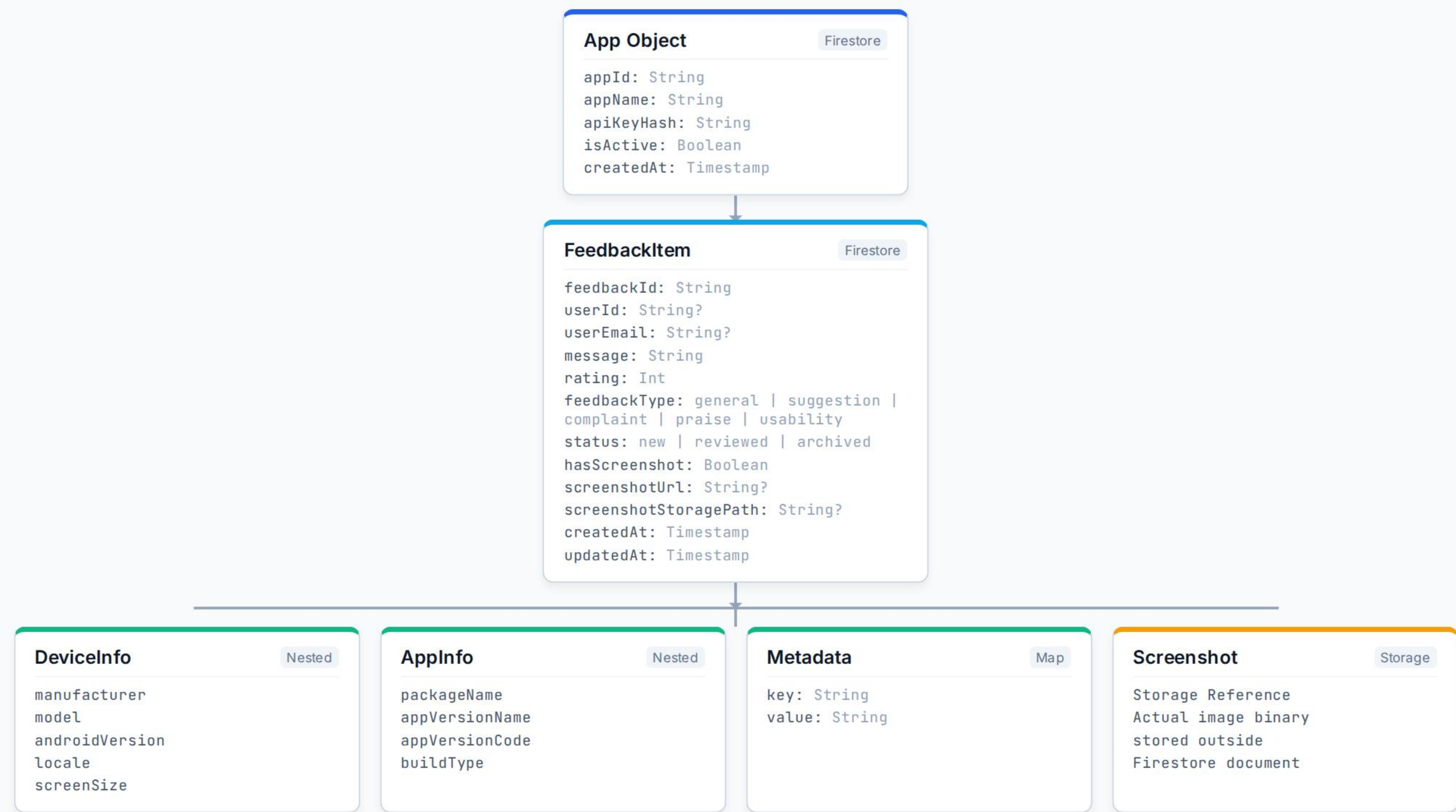
updateFeedbackStatus

Modifies the status field (e.g., 'new' to 'reviewed') for feedback management.

getFeedbackByFilters

Retrieves feedback items for the portal using filters such as status, feedback type, rating, app version, device model, and date range.

Firebase Data Model Structure



Firestore: `apps/{appId}/feedback/{feedbackId}` | Storage: `feedback-screenshots/{appId}/{feedbackId}/screenshot.jpg`

Efficient Data Storage

Structured Firestore Data

- App object stores app identity and API key hash.
- FeedbackItem stores searchable fields such as status, rating, feedbackType, and createdAt.
- DeviceInfo, AppInfo, and Metadata are nested directly inside the feedback document.
- Screenshot files are not stored inside Firestore to preserve database performance.

Efficient Storage Process

01

Feedback Submission

The SDK collects message, rating, feedback type, device info, app info, metadata, and optional screenshot.

02

Structured Firestore Document

The backend stores the feedback item under:
`apps/{appId}/feedback/{feedbackId}`

03

Separate Screenshot Storage

The screenshot is compressed and uploaded to Storage:
`feedback-screenshots/{appId}/{feedbackId}/screenshot.jpg`

04

Lightweight Reference

Firestore stores only screenshotUrl and path, keeping structured data fast to query.

Efficient Feedback Querying

The portal remains fast by using scoped Firestore queries, indexed fields, pagination, and lazy loading.

`appId`

Feedback Separation

Feedback items are scoped by app. This allows the portal to load only items that belong to the selected app and supports clear security separation across tenants.

`feedbackType + rating`

Product Insight Filtering

Indexes allow fast filtering by feedback type and rating, such as tracking low-rated usability feedback or compiling positive feature praise.

`createdAt`

Time-Based Pagination

Feedback items are sorted from newest to oldest. The portal loads the latest 20 items first and continues with cursor-based pagination for rapid loads.

Web Portal Functions

The portal allows product teams and developers to view, filter, and manage user feedback collected from the SDK.

 login

 viewDashboard

 listFeedback

 filterFeedback

 searchFeedback

 openFeedbackDetails

 viewScreenshot

 viewDeviceInfo

 viewAppInfo

 viewMetadata

 updateFeedbackStatus

 markAsReviewed

 exportFeedback

Supported Applications

Feedback SDK can be seamlessly integrated into almost any Android application that requires structured user feedback, ratings, and rich contextual data.



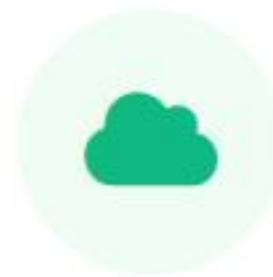
E-commerce Apps

Track user friction during checkout flows.



Banking & Fintech

Securely collect usability feedback.



SaaS Mobile Apps

Collect product feedback from active users.



Beta Testing Apps

Capture usability insights automatically.



Education Apps

Gather feature requests seamlessly.



Internal Enterprise Apps

Log UX suggestions for internal tools.